

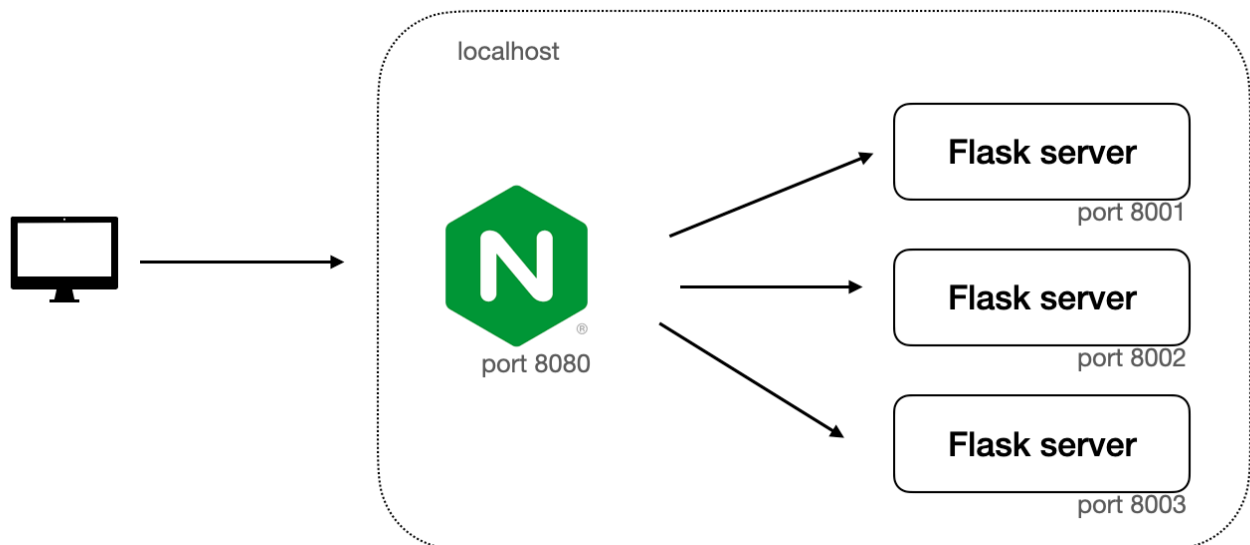
A Step-by-Step Guide to Distributing Traffic Across Web Servers

 systemdesignschool.io/fundamentals/load-balancing-code-lab



Load balancing Codelab

Let's build a quick demo using NGINX to distribute traffic among three web servers.



Using NGINX as a load balancer

To follow along with this demo on your computer, you'll need to have Flask and NGINX installed.

Creating a Simple Flask Web Server

First, let's create a simple Flask web server that returns "Hello, {port number}!" as a response:

```
# demo.py from flask import Flask import sys app = Flask('demo') @app.route('/')
def hello_world(): return f'Hello, {app.config["PORT"]}!\n' if __name__ ==
'__main__': port = int(sys.argv[1]) if len(sys.argv) > 1 else 8000
app.config['PORT'] = port # Set the port in app config app.run(host='127.0.0.1',
port=port)
```

Now, we can start three server instances:

```
python demo.py 8001 python demo.py 8002 python demo.py 8003
```

Configuring NGINX as a Load Balancer

Next, we will configure NGINX as a load balancer to distribute traffic among the three web servers. Create a new configuration file `/etc/nginx/conf.d/demo.conf`:

```
upstream demo { server localhost:8001; server localhost:8002; server
localhost:8003; } server { listen 8080; location / { proxy_pass http://demo; } }
```

Restart the NGINX service using `sudo systemctl restart nginx`.

Now, you should be able to access `http://localhost:8080/`. By default, the load balancing method is round-robin, so the responses from the load balancer should be "Hello, 8001!", "Hello, 8002!", and "Hello, 8003!" in sequence.

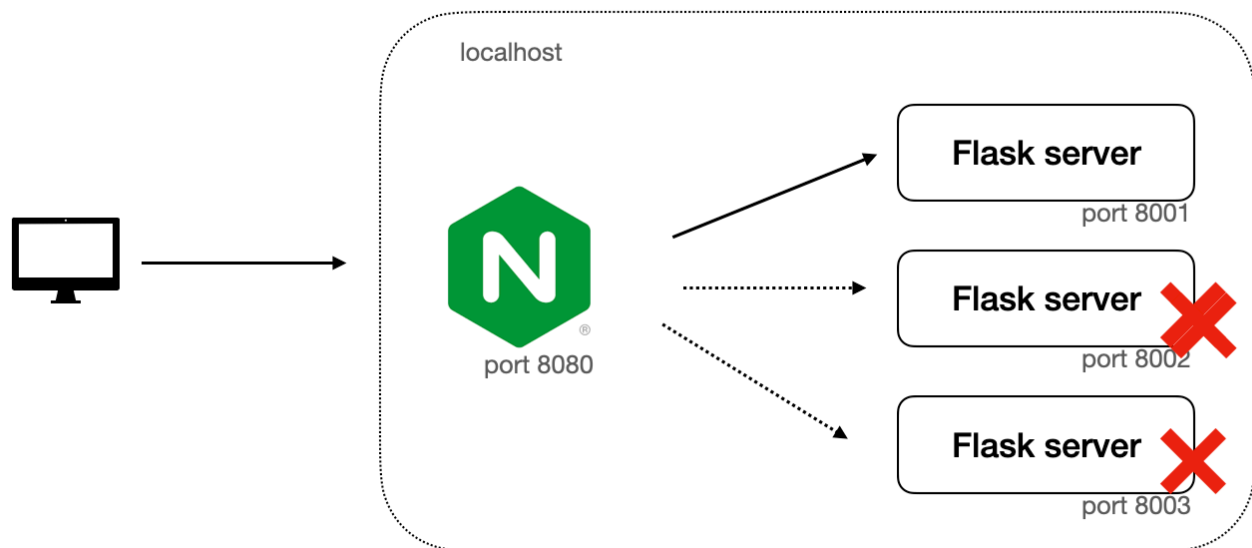
Video Demonstration

For a visual walkthrough of this demonstration on Debian 11, refer to the following video:

[NGINX Load Balancing Demo](#)

Load Balancer and High Availability

To emulate servers going offline, we stop the web server that runs on 8002 and 8003 and visit our load balancer at `[http://localhost:8080/](http://localhost:8080/)`. It should still be accessible and always return `Hello, 8001!`, the message returned by the first server. This is why load balancers and high availability are important: two web servers are down, and we are still serving requests from the remaining one without the user noticing any downtime. We can scale a service by adding more web servers and putting them behind load balancers.



Using NGINX as a load balancer with crashed servers

A Note on Layer 4 vs Layer 7 Load Balancing

If we get into the details of computer networking, there are two ways to load balance in terms of [OSI](#) networking model - Layer 4 and Layer 7 load balancing. The one we just discussed is Layer 7 load balancing.

Layer 4 is the Transport Layer. Layer 4 load balancing is done on the packet-level and requires access to lower level routing devices. Layer 7 is Application Layer load balancing and is done entirely on the HTTP level. There are pros and cons of load balancing at each level. If you are interested, you can read more about it [here](#).

Software engineers mostly deal with Layer 7 since web applications we develop live on Layer 7. So don't worry about it if you don't understand the lower level computer networking terminologies (that's more of the jobs of network engineers and maybe SREs). In the following sections, we will explore the Layer 7 load balancing with Nginx.